

A Parallel Algorithm for Longest Common Subsequence Problem (LCS) on Parallel External Memory (PEM) Model

*A M. Tech Project Report Submitted
in Fulfillment of the Requirements
for the Degree of*

Master of Technology

by

Ashish Verma

(244101006)

under the guidance of

Prof. Sajith Gopalan



to the

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI
GUWAHATI - 781039, ASSAM**

CERTIFICATE

*This is to certify that the work contained in this thesis entitled “**A Parallel Algorithm for Longest Common Subsequence Problem (LCS) on Parallel External Memory (PEM) Model** ” is a bonafide work of **Ashish Verma (Roll No. 244101006)**, carried out in the Department of Computer Science and Engineering, Indian Institute of Technology Guwahati under my supervision and that it has not been submitted elsewhere for a degree.*

Supervisor: **Prof. Sajith Gopalan**

Professor,

Jun, 2026

Guwahati.

Department of Computer Science & Engineering,

Indian Institute of Technology Guwahati, Assam.

Acknowledgements

I would like to take this opportunity to express my heartfelt gratitude to my thesis supervisor, Prof. Sajith Gopalan, from the Department of Computer Science and Engineering at IIT Guwahati, for his invaluable guidance, constant support, and encouragement throughout this project. His insightful advice and continuous efforts have not only guided me in this work but also imparted valuable lessons that will assist me in my future career.

I am also deeply grateful to my parents and friends for their unwavering support and cooperation. Their wisdom and encouragement have been a constant source of strength in my life. Finally, I would like to reiterate my sincere thanks to Prof. Sajith Gopalan, without whose guidance this project would have remained a distant dream.

Contents

List of Figures	vii
List of Tables	ix
1 Introduction	1
1.1 Problem Definition: Longest Common Subsequence	1
1.2 The Parallel External Memory (PEM) Model	2
1.3 Motivation	2
1.4 Objectives	3
1.5 Organization of The Report	4
2 Literature Review	5
2.1 Standard Sequential Algorithms (The RAM Model)	5
2.2 Sequential External Memory Algorithms (Blocked DP)	6
2.3 Parallel Algorithms (The PRAM Model)	7
2.4 The Gap in Existing Literature	8
2.5 Statement of Novelty: Our Approach	8
2.5.1 Coarse-Grained Wavefront Scheduling	9
2.5.2 Managing CREW Constraints with Overlapping Dependencies	9
2.6 Summary	9

3	Proposed Solution: The Tiled Wavefront Algorithm	11
3.1	Algorithm Design and Grid Partitioning	11
3.1.1	Sizing the Tile to Internal Memory	12
3.1.2	The Logical Partition	12
3.2	Step-by-Step Execution	12
3.2.1	Phase 1: Boundary Initialization (I/O Phase)	12
3.2.2	Phase 2: Wavefront Scheduling	13
3.2.3	Phase 3: Local Tile Computation (The PRAM-EM Hybrid Phase) .	14
3.3	Detailed Numerical Trace	16
3.3.1	Key Observations from the Trace	16
4	Rigorous Complexity Analysis	19
4.1	The WT Scheduling Principle (Brent's Theorem)	19
4.1.1	Why Addition?	20
4.2	Derivation of the I/O Bounds	20
4.2.1	Step 1: Calculating Total I/O Work (W)	21
4.2.2	Step 2: Calculating the Critical Path (T_∞)	21
4.2.3	Step 3: Applying the Bounds	22
4.3	Interpretation of the Complexity Bound	22
5	Conclusion and Future Work	25
5.1	Conclusion	25
5.2	Future Work	26
	References	29

List of Figures

3.1 A 4×4 Logical Grid showing 7 independent Diagonal Waves (W_1 to W_7).

Tiles with the same wave number can be processed in parallel. 13

List of Tables

2.1	Comparison showing the gap in existing LCS algorithms.	10
3.1	Parallel Processor Schedule. Notice how Wave 4, having more tiles than processors, is automatically divided into two rounds to balance the load. .	16

Chapter 1

Introduction

This study focuses on designing an efficient parallel algorithm for the Longest Common Subsequence (LCS) problem, a fundamental challenge in computer science, specifically tailored for the Parallel External Memory (PEM) computational model [SPS23]. While the LCS problem has been solved sequentially and on theoretical parallel models, processing massive strings that exceed the capacity of a computer's main memory presents unique challenges that require a specialized approach.

1.1 Problem Definition: Longest Common Subsequence

Before discussing the computational models, it is essential to formally define the core problem we are solving.

Let a *sequence* (or string) be an ordered list of characters. A *subsequence* of a given sequence is formed by deleting zero or more characters from the original sequence without changing the order of the remaining characters. For example, if we have a string "ABCDEF", then "ACE" and "BDF" are valid subsequences.

The **Longest Common Subsequence (LCS)** problem asks us to find the longest subsequence that is present in two or more given strings[CLRS09]. For example, given two strings:

- $X = \text{"AGGTAB"}$
- $Y = \text{"GXTXAYB"}$

The longest common subsequence is "GTAB", which has a length of 4. Finding this length efficiently becomes computationally demanding as the sizes of the strings grow.

1.2 The Parallel External Memory (PEM) Model

Traditional algorithms are designed for the standard RAM model, which assumes that every piece of data can be accessed instantly. However, modern computers have a memory hierarchy. Fast internal memory (RAM) is limited in size, while the external memory (Hard Disks or SSDs) is massive but extremely slow.

When the input strings are so large that they cannot fit into the internal memory, the computer must constantly transfer chunks of data back and forth from the slow disk. This data transfer is called an **I/O (Input/Output) operation**. I/O operations are the slowest part of any modern computing system.

The **Parallel External Memory (PEM)** model accurately represents this reality for multi-core systems[SPS23]. It is characterized by three main parameters:

- **P**: The number of synchronous processors available for computation.
- **M**: The size of the fast, private internal memory available to *each* processor.
- **B**: The block size. Data is transferred between the external memory and the internal memory in contiguous blocks of size B , rather than one element at a time.

Our goal on the PEM model is not just to divide the computational work among P processors, but to strictly minimize the number of B -sized block transfers (I/O rounds).

1.3 Motivation

The Longest Common Subsequence problem is not just a theoretical exercise; it has critical, large-scale applications in various real-world domains where datasets are massive.

- **Bioinformatics and Computational Biology:** LCS is the foundational logic behind sequence alignment tools (such as BLAST). Biologists use these algorithms to compare DNA, RNA, and protein sequences to find genetic similarities. Modern whole-genome datasets are often hundreds of gigabytes or even terabytes in size. These datasets far exceed the main memory of any single machine, making I/O bottlenecks a severe issue.
- **Version Control and Data Synchronization:** File comparison utilities, such as the *diff* command in Unix or the underlying merging logic in version control systems like Git, use LCS-based algorithms to find the exact differences between files. When dealing with massive database dumps or large software repositories, calculating these "patches" efficiently requires minimizing disk access.

As processor speeds approach their inherent physical limits, using multiple processors is essential for achieving speedups. However, for problems involving massive datasets, simply adding more processors is not enough. If multiple processors constantly request small pieces of data from the disk, the I/O bus becomes heavily congested. The goal of this research is to address this specific challenge: leveraging the parallel processing capabilities of the PEM model to create an algorithm that is mathematically optimized to reduce I/O bottlenecks.

1.4 Objectives

The primary objective of this thesis is to design, analyze, and formalize an I/O-efficient parallel algorithm for the Longest Common Subsequence problem on the PEM model. The specific objectives are:

- To conduct a detailed literature survey of existing LCS algorithms, highlighting the limitations of standard sequential DP, PRAM (Parallel RAM) algorithms, and sequential blocked DP approaches.
- To design a novel **Tiled Wavefront Algorithm** optimized for the Concurrent Read Exclusive Write (CREW) PEM model, which groups computational work to fit entirely within the internal memory M .
- To formally analyze the algorithm's parallel I/O complexity using the WT Scheduling Principle (Brent's Theorem for the PEM model).
- To theoretically prove that this new algorithm achieves an optimal balance between parallel computation speedup and I/O minimization.

1.5 Organization of The Report

This chapter introduced the formal definition of the Longest Common Subsequence (LCS) problem and the Parallel External Memory (PEM) model. We also outlined the motivation for this research, driven by applications in bioinformatics and large-scale data synchronization, and stated the specific objectives of this thesis.

The remainder of this thesis is organized as follows:

- **Chapter 2** provides a detailed literature survey. It reviews the evolution of LCS algorithms across different computational models and establishes the algorithmic novelty of the proposed research by identifying the gap in existing parallel I/O approaches.
- **Chapter 3** presents the proposed Proposed Solution. It details the Tiled Wavefront Algorithm, explains the logical grid partitioning, and provides a step-by-step trace of how the processors compute the dynamic programming matrix in diagonal waves.
- **Chapter 4** focuses on the Rigorous Complexity Analysis. It mathematically derives the total I/O work and the critical path latency, applying Brent's Theorem to

establish the final upper bounds for the algorithm on the PEM model.

- **Chapter 5** concludes the thesis, summarizing the contributions and discussing potential avenues for future research and practical implementations.

Chapter 2

Literature Review

The design of any algorithmic solution is deeply influenced by the underlying hardware model it is built for. The Longest Common Subsequence (LCS) problem has been studied extensively over the past few decades. However, existing solutions are highly specialized for specific environments.

This chapter reviews the evolution of algorithms for the LCS problem across three distinct computational paradigms: the standard RAM model, the External Memory (EM) model, and the Parallel Random Access Machine (PRAM) model. By analyzing the strengths and limitations of these existing methods, we will clearly identify the current gap in the research and establish the algorithmic novelty of the proposed solution on the Parallel External Memory (PEM) model.

2.1 Standard Sequential Algorithms (The RAM Model)

The classic approach to solving the LCS problem uses Dynamic Programming (DP). For two strings of length N , the algorithm creates an $N \times N$ matrix. The value of any cell in this matrix depends on the values of its top, left, and top-left diagonal neighbors[CLRS09, AHU74].

$$L[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ L[i-1][j-1] + 1 & \text{if } X[i-1] = Y[j-1] \\ \max(L[i-1][j], L[i][j-1]) & \text{otherwise} \end{cases}$$

This sequential algorithm completes the task in $O(N^2)$ time. In the theoretical Random Access Machine (RAM) model, this is considered optimal in terms of computational work. The RAM model operates on a very simple assumption: every single memory access takes exactly $O(1)$ time, regardless of how large the data is.

Limitation: In the real world, the RAM assumption fails when dealing with massive datasets, such as whole-genome sequences in bioinformatics. When the $N \times N$ DP table becomes too large to fit into the computer's fast internal memory, the operating system is forced to swap data back and forth to the slow external hard disk. Because the standard sequential algorithm computes the table row by row (or cell by cell), it constantly requests data that is not currently in the fast memory. This leads to a severe performance degradation known as "disk thrashing." As a result, the execution time is completely dominated by the slow disk reads and writes, making the standard RAM approach impractical for massive strings.

2.2 Sequential External Memory Algorithms (Blocked DP)

To address the disk thrashing problem, researchers developed algorithms based on the External Memory (EM) model (proposed by Aggarwal and Vitter)[AV88]. The EM model explicitly recognizes the memory hierarchy: it accounts for a slow infinite disk and a fast, limited internal memory of size M . Crucially, data is transferred between them in contiguous chunks called blocks, of size B .

To optimize the LCS problem for the EM model, researchers use a well-known technique called **Blocked Dynamic Programming** (or Tiled DP). Instead of computing the DP

table cell by cell, the EM approach divides the large $N \times N$ grid into smaller square blocks (or tiles) of size $S \times S$. The algorithm is designed so that an entire block, along with its required boundaries, fits perfectly inside the internal memory M .

The processor loads a block, performs all S^2 local calculations purely in the fast RAM without any disk interruptions, and then writes only the newly computed boundaries back to the disk. This drastically reduces the number of slow I/O block transfers.

Limitation: While Blocked DP successfully minimizes I/O transfers, it is strictly a sequential approach designed for a single processor ($P = 1$). The standard EM model does not account for multiple processors working simultaneously. Therefore, it cannot take advantage of modern multi-core architectures to speed up the massive $O(N^2)$ computational work.

2.3 Parallel Algorithms (The PRAM Model)

To speed up the actual computation time, researchers use the Parallel Random Access Machine (PRAM) model[Já92]. The PRAM model assumes we have P processors that can all work at the same time.

The standard parallel solution for the LCS problem uses the **Wavefront Method**. Because the calculation of a DP cell depends on its top and left neighbors, we cannot compute the entire matrix simultaneously. However, all cells lying on the same anti-diagonal line (a "wavefront") are completely independent of each other. PRAM algorithms assign individual processors to compute individual cells along these diagonal waves concurrently.

Limitation: The PRAM model assumes a "flat" memory hierarchy. It assumes all P processors can access any piece of shared memory instantly, without any penalties for block transfers. If we try to run a standard PRAM wavefront algorithm on a system where the strings are stored on an external disk, it causes a catastrophe. Every single processor will simultaneously request tiny, scattered fragments of data (individual characters or boundary cells) from the disk. Because the disk must transfer data in blocks of size B , this creates

extreme I/O congestion. The system becomes completely overwhelmed trying to fetch countless blocks, causing the processors to sit idle.

2.4 The Gap in Existing Literature

The literature review reveals a clear separation in how the LCS problem is handled:

- We know how to optimize for slow disk I/O using Blocked DP, but only for a single processor.
- We know how to optimize for computation time using the parallel Wavefront method, but it causes extreme I/O penalties.

What is missing from the existing literature is a unified algorithmic approach that handles both massive parallelism and external memory constraints simultaneously. This is exactly what the **Parallel External Memory (PEM)** model is designed to address. The PEM model requires us to simultaneously manage the number of processors (P), the limited local memory of each processor (M), and the block transfer size (B).

2.5 Statement of Novelty: Our Approach

The core novelty of this thesis is not the invention of blocked DP, nor is it the invention of the wavefront method. **The novelty lies in successfully designing a scheduling algorithm that combines them under the strict constraints of the shared-memory PEM model.**

Applying Blocked DP in a parallel environment is mathematically non-trivial. Our proposed **Tiled Wavefront Algorithm** introduces the following novel solutions:

2.5.1 Coarse-Grained Wavefront Scheduling

We elevate the parallel wavefront concept from the *cell level* to the *block level*. In our algorithm, processors do not compute individual DP cells; they are assigned entire $S \times S$ independent sub-matrices. By mathematically sizing S based on the internal memory limit M , we ensure that each processor performs a massive amount of local computational work. This effectively "hides" the latency of fetching blocks from the external memory, explicitly balancing the parallel computation time with the I/O cost.

2.5.2 Managing CREW Constraints with Overlapping Dependencies

In sequential blocked DP, managing the boundaries between blocks is simple because only one processor accesses the disk. In our parallel approach, multiple processors work on the same diagonal wave simultaneously, sharing the same external memory. Our algorithm is specifically designed to adhere to the **Concurrent Read Exclusive Write (CREW)** constraints of the PEM model:

- **Concurrent Read (CR):** Two adjacent tiles on a diagonal wave often need to read the same boundary data generated by a tile from a previous wave. Our algorithm explicitly leverages the CR property to allow processors to fetch these overlapping boundary blocks concurrently without queuing delays.
- **Exclusive Write (EW):** If multiple processors attempt to write their results back to the same memory block, it causes a write conflict. Our logical grid partitioning guarantees that every processor writes its newly computed boundary to a strictly distinct block in the shared memory, perfectly satisfying the EW constraint.

2.6 Summary

While blocked dynamic programming is a known sequential optimization, existing literature does not provide a mechanism to safely and efficiently distribute those blocks across multiple

processors sharing a single external disk. Our approach bridges this gap. It provides a conflict-free scheduling mechanism that achieves optimal parallel speedup while strictly minimizing I/O block transfers.

Computational Model	Parallel ($P > 1$)?	I/O Aware (B, M)?	Main Bottleneck
Standard RAM	No	No	Severe Disk Thrashing
PRAM (Wavefront)	Yes	No	Extreme I/O Congestion
EM Model (Blocked DP)	No	Yes	Slow Computation Time
PEM (Our Approach)	Yes	Yes	None (Optimal Balance Achieved)

Table 2.1 Comparison showing the gap in existing LCS algorithms.

Chapter 3

Proposed Solution: The Tiled Wavefront Algorithm

As established in the literature review, standard PRAM algorithms assign one processor per Dynamic Programming (DP) cell, which leads to a massive I/O bottleneck. Conversely, Sequential External Memory algorithms process large blocks to save I/O but cannot utilize multiple processors.

To bridge this gap, this chapter proposes the **Tiled Wavefront Algorithm**. This algorithm is specifically designed to adhere to the strict rules of the Concurrent Read Exclusive Write (CREW) Parallel External Memory (PEM) model. The core intuition is to group the computational work into large, independent sub-matrices (tiles) that fit entirely within a single processor's fast internal memory, and then schedule the parallel computation of these tiles across diagonal waves.

3.1 Algorithm Design and Grid Partitioning

Let X and Y be the input strings, both of length N , stored compactly in the shared external disk.

3.1.1 Sizing the Tile to Internal Memory

Instead of dealing with individual characters, we divide the logical $N \times N$ DP table into square tiles of dimension $S \times S$. To compute a single tile, a processor requires three pieces of data:

1. Substrings of X and Y corresponding to the tile (size $2S$).
2. The computed bottom boundary from the tile directly above it (size S).
3. The computed right boundary from the tile directly to its left (size S).

To execute the standard $O(S^2)$ LCS calculation[CLRS09] inside a tile entirely in RAM without any disk swapping, the total data must fit within the internal memory parameter M . Therefore, we explicitly set the tile size parameter as $S = \Theta(\sqrt{M})$.

3.1.2 The Logical Partition

By dividing strings X and Y into contiguous chunks of length S , we effectively partition the vast DP table into a logical grid of $K \times K$ tiles, where $K = \lceil N/S \rceil$. Each tile, denoted as $T(i, j)$, represents an independent $S \times S$ sub-problem.

3.2 Step-by-Step Execution

The algorithm executes in three distinct phases: Initialization, Wavefront Scheduling, and Local Tile Computation.

3.2.1 Phase 1: Boundary Initialization (I/O Phase)

The base case for LCS dictates that if either string is empty, the LCS length is 0. In our logical grid, this corresponds to initializing the 0^{th} row and 0^{th} column of the external memory boundary arrays to 0.

Because we write these $O(N)$ zeros in contiguous blocks of size B , this phase requires $O(N/B)$ I/O operations. Under the CREW constraint, the P processors evenly divide these blocks. Since processors are assigned distinct blocks to write to, there are strictly no concurrent write conflicts.

3.2.2 Phase 2: Wavefront Scheduling

The state of any tile $T(i, j)$ depends entirely on the tiles $T(i-1, j)$ and $T(i, j-1)$. Because of these dependencies, the $K \times K$ grid cannot be computed randomly. We must process the tiles in **diagonal waves**.

As shown in Figure 3.1, all tiles lying on the same diagonal line have their dependencies completely resolved in previous waves. Therefore, all tiles on a given diagonal are mathematically independent and can be computed concurrently by our P processors.

String Y (Divided into chunks of size S)

String X (Chunks of size S)

W_1	W_2	W_3	W_4
W_2	W_3	W_4	W_5
W_3	W_4	W_5	W_6
W_4	W_5	W_6	W_7

Fig. 3.1 A 4×4 Logical Grid showing 7 independent Diagonal Waves (W_1 to W_7). Tiles with the same wave number can be processed in parallel.

The total number of diagonal waves for a $K \times K$ grid is exactly $2K - 1$. The algorithm iterates through each wave d (where d ranges from 1 to $2K - 1$), assigning the independent tiles to the available processors.

3.2.3 Phase 3: Local Tile Computation (The PRAM-EM Hybrid Phase)

For a given wave, the independent tiles are distributed among the processors. For each assigned tile, a processor executes three strict steps:

- **Read Phase (I/O):** The processor fetches the S -length substring chunks, along with the boundaries from the top and left tiles. Because this is a Concurrent Read (CR) model, adjacent tiles on the same wave can read the same corner boundary simultaneously without queuing.
- **Compute Phase (CPU):** The processor calculates the $S \times S$ LCS matrix internally. By isolating the computation inside the fast internal memory, the processor can run the standard sequential DP algorithm at maximum speed. *Zero external I/O occurs during this phase*, meaning the processor never has to pause to wait for the slow disk until the entire tile is completely finished.
- **Write Phase (I/O):** The processor extracts only the newly computed bottom row and rightmost column of the tile and writes them back to external memory. Because processors are assigned entirely different tiles, their output boundaries map to different blocks. This perfectly satisfies the Exclusive Write (EW) condition.

This strict separation between reading, computing, and writing is what gives the Tiled Wavefront Algorithm its power. By grouping thousands of small calculations into a few large block transfers, the algorithm effectively hides the high latency of the external disk. This ensures that the expensive processors spend their time actively computing answers rather than sitting idle waiting for data.

Algorithm 1 Tiled Wavefront LCS on the PEM Model

Require: Strings X, Y of length N in External Memory (EM); P processors; Internal Memory capacity M ; Block size B .

Ensure: The computed boundaries of the Longest Common Subsequence (LCS) matrix in EM.

▷ **Phase 1: Initialization**

- 1: $S \leftarrow \Theta(\sqrt{M})$ ▷ Size tile to fit fully in internal memory
- 2: $K \leftarrow \lceil N/S \rceil$ ▷ Logical grid is $K \times K$ tiles
- 3: **for all** $k \leftarrow 1$ **to** N **in parallel using** P **processors do**
- 4: $C[k, 0] \leftarrow 0$
- 5: $C[0, k] \leftarrow 0$ ▷ Initialize base cases directly in EM
- 6: **end for**

▷ **Phase 2: Wavefront Scheduling**

- 7: **for** $d \leftarrow 1$ **to** $2K - 1$ **do** ▷ Iterate over every diagonal wave
- 8: $W_d \leftarrow \{T(i, j) \mid i + j - 1 = d, \text{ for } 1 \leq i, j \leq K\}$

▷ **Phase 3: Local Tile Computation**

- 9: **for each** tile $T(i, j) \in W_d$ **in parallel using** P **processors do**
 - 10: Read string chunks $X[(i - 1)S + 1 \dots iS]$ and $Y[(j - 1)S + 1 \dots jS]$ from EM
 - 11: Read top boundary from tile $T(i - 1, j)$ from EM
 - 12: Read left boundary from tile $T(i, j - 1)$ from EM
 - 13: Compute $S \times S$ LCS matrix for $T(i, j)$ strictly in local internal memory
 - 14: Write newly computed bottom boundary of $T(i, j)$ to EM
 - 15: Write newly computed right boundary of $T(i, j)$ to EM
 - 16: **end for**
 - 17: **end for**
-

3.3 Detailed Numerical Trace

To clearly illustrate how the Tiled Wavefront Algorithm adapts to limited hardware, we trace its execution using concrete parameters.

Assume a highly constrained PEM environment where:

- Processors (P) = 3
- Internal Memory (M) = 16 words (allowing a tile size of $S = 4$)
- Block Size (B) = 2 words

We are given two strings, X and Y , both of length $N = 16$. Because $S = 4$, the strings are divided into 4 chunks, generating a 4×4 logical grid containing exactly 16 tiles (as seen in Figure 3.1). There are $2(4) - 1 = 7$ waves. Table 3.1 demonstrates the parallel schedule.

Wave (d)	Active Tiles	Processor 1	Processor 2	Processor 3
Wave 1	$T(1, 1)$	$T(1, 1)$	<i>Idle</i>	<i>Idle</i>
Wave 2	$T(1, 2), T(2, 1)$	$T(1, 2)$	$T(2, 1)$	<i>Idle</i>
Wave 3	$T(1, 3), T(2, 2), T(3, 1)$	$T(1, 3)$	$T(2, 2)$	$T(3, 1)$
Wave 4 (Round a)	<i>4 tiles available</i>	$T(1, 4)$	$T(2, 3)$	$T(3, 2)$
Wave 4 (Round b)		$T(4, 1)$	<i>Idle</i>	<i>Idle</i>
Wave 5	$T(2, 4), T(3, 3), T(4, 2)$	$T(2, 4)$	$T(3, 3)$	$T(4, 2)$
Wave 6	$T(3, 4), T(4, 3)$	$T(3, 4)$	$T(4, 3)$	<i>Idle</i>
Wave 7	$T(4, 4)$	$T(4, 4)$	<i>Idle</i>	<i>Idle</i>

Table 3.1 Parallel Processor Schedule. Notice how Wave 4, having more tiles than processors, is automatically divided into two rounds to balance the load.

3.3.1 Key Observations from the Trace

1. Massive I/O Savings: In Wave 1, Processor 1 reads 8 blocks of data and writes 4 blocks. In return, it performs $S^2 = 16$ internal DP calculations. As M scales to realistic

sizes (e.g., $M = 1,000,000$ words, $S = 1000$), a processor performs one million computations for only a few hundred block transfers, virtually eliminating the I/O bottleneck.

2. Automatic Load Balancing: The algorithm gracefully handles the "bulk" of the matrix. In Wave 4, there are 4 independent tiles but only 3 processors. The schedule automatically breaks this into two rounds (4a and 4b). This mechanism forms the physical basis for Brent's Theorem, which we will use in Chapter 4 to mathematically prove the algorithm's optimal performance.

Chapter 4

Rigorous Complexity Analysis

Proving the efficiency of an algorithm on the Parallel External Memory (PEM) model requires a different approach than traditional sequential models. In the standard RAM model, complexity is analyzed by counting the number of CPU instructions. However, in the PEM model, CPU operations performed entirely inside the fast internal memory are considered "free" relative to the immense cost of fetching data from the disk.

Therefore, the performance of the Tiled Wavefront Algorithm is strictly measured by its **Parallel I/O Complexity**: the total number of parallel block transfers required between the shared external memory and the local internal memories of the processors. This chapter rigorously derives the exact mathematical upper bounds for this metric.

4.1 The WT Scheduling Principle (Brent's Theorem)

To analyze parallel algorithms where the amount of available work changes over time (such as our wavefront expanding and shrinking), we utilize the WT Scheduling Principle, which is the PEM model's equivalent of Brent's Theorem[Já92, SPS23].

Brent's Theorem states that the total parallel time T_P (in our case, parallel I/O rounds)

required to execute an algorithm on P processors is bounded by the following inequality:

$$T_P \leq T_\infty + \frac{W}{P}$$

This formula elegantly combines two distinct theoretical bounds:

- **Total Work (W):** The total number of I/O operations required if the entire algorithm were executed by a *single* processor.
- **Critical Path Latency (T_∞):** The minimum number of sequential I/O steps required to complete the algorithm if we had an *infinite* number of processors. This represents the longest chain of unavoidable data dependencies.

4.1.1 Why Addition?

A common question in parallel analysis is why T_∞ and W/P are added together. During execution, the algorithm exists in one of two states. In **State 1**, there are fewer active tiles than processors (such as the top-left and bottom-right corners of our grid). Processors are forced to sit idle, and the time taken is strictly bound by the critical dependency path T_∞ . In **State 2**, there is a massive bulk of independent work (the wide middle diagonals of the grid). All P processors are active, and the work W is divided perfectly among them (W/P). Brent's Theorem safely adds the worst-case time of both states to guarantee a rigorous upper bound.

4.2 Derivation of the I/O Bounds

We now apply this theorem systematically to the proposed Tiled Wavefront Algorithm. Let X and Y be input strings of length N . Let M be the internal memory size, B be the block transfer size, and S be the tile dimension such that $S = \Theta(\sqrt{M})$. The logical grid has dimension $K \times K$, where $K = \lceil N/S \rceil$.

4.2.1 Step 1: Calculating Total I/O Work (W)

To find W , we must determine the I/O cost of processing the entire grid sequentially. First, we calculate the cost to process a single $S \times S$ tile. A processor must load:

1. The corresponding substring chunks of X and Y (size $2S$).
2. The computed bottom boundary from the tile above (size S).
3. The computed right boundary from the tile to the left (size S).

The total volume of data read and written per tile is proportional to $O(S)$. Because this data is stored continuously in external memory, transferring $O(S)$ elements requires exactly $O(S/B)$ block I/O operations.

The total number of tiles in the grid is K^2 . Therefore, the total work W across all tiles is:

$$W = K^2 \cdot O\left(\frac{S}{B}\right)$$

Substituting $K = N/S$, we expand the equation:

$$W = \left(\frac{N}{S}\right)^2 \cdot O\left(\frac{S}{B}\right) = \frac{N^2}{S^2} \cdot O\left(\frac{S}{B}\right) = O\left(\frac{N^2}{SB}\right)$$

4.2.2 Step 2: Calculating the Critical Path (T_∞)

The critical path is determined by the maximum depth of the dependency graph. Because tile $T(i, j)$ strictly depends on tiles $T(i - 1, j)$ and $T(i, j - 1)$, the longest sequence of dependencies stretches from the top-left tile $T(1, 1)$ down to the bottom-right tile $T(K, K)$.

This path precisely corresponds to the number of diagonal waves. A $K \times K$ grid has exactly $2K - 1$ diagonal waves. If we had infinite processors, all tiles on a single wave would be processed simultaneously in $O(S/B)$ I/O rounds. Thus, the critical path latency

is the number of waves multiplied by the cost per wave:

$$T_\infty = (2K - 1) \cdot O\left(\frac{S}{B}\right) = O\left(K \cdot \frac{S}{B}\right)$$

Substituting $K = N/S$, we find the critical path bound:

$$T_\infty = O\left(\frac{N}{S} \cdot \frac{S}{B}\right) = O\left(\frac{N}{B}\right)$$

4.2.3 Step 3: Applying the Bounds

We now substitute our derived values for total work (W) and critical path (T_∞) back into Brent's formula ($T_P \leq T_\infty + \frac{W}{P}$):

$$T_P \leq O\left(\frac{N}{B}\right) + \frac{O\left(\frac{N^2}{SB}\right)}{P}$$

Rearranging the terms for clarity, we get:

$$T_P = O\left(\frac{N^2}{P \cdot S \cdot B} + \frac{N}{B}\right)$$

Finally, we substitute our optimal tile sizing parameter $S = \Theta(\sqrt{M})$. This guarantees that each tile perfectly fills the processor's internal memory without overflowing. The substitution yields the final parallel I/O complexity bound:

$$\mathbf{T_P} = \mathbf{O}\left(\frac{\mathbf{N^2}}{\mathbf{PB\sqrt{M}}} + \frac{\mathbf{N}}{\mathbf{B}}\right)$$

4.3 Interpretation of the Complexity Bound

The derived mathematical bound proves the efficiency of the proposed algorithm and perfectly models its physical execution on the hardware.

The first term, $O\left(\frac{N^2}{PB\sqrt{M}}\right)$, represents the perfectly parallelized "bulk" phase of the

algorithm. It shows that our total sequential I/O work is successfully divided by P (the speedup from using multiple processors) and reduced by a factor of B (the speedup from block transfers). Most importantly, it is reduced by a massive factor of $\sqrt{M}$ due to our tiling strategy, proving that grouping computations in internal memory eliminates disk thrashing.

The second term, $O\left(\frac{N}{B}\right)$, represents the inescapable latency constraint. Even with infinite processors, a computer must still physically read the length of the string N from the external disk in blocks of size B . This term dominates only when the strings are relatively small or the number of processors P is exceptionally large.

Together, this rigorous analysis proves that the Tiled Wavefront Algorithm achieves mathematically optimal I/O scaling under the Concurrent Read Exclusive Write (CREW) Parallel External Memory model.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

As datasets in fields like bioinformatics and data synchronization grow to massive scales, traditional algorithmic models that assume uniform memory access are no longer sufficient. The Longest Common Subsequence (LCS) problem serves as a perfect example of this challenge. While sequential dynamic programming is computationally optimal, it suffers from severe disk thrashing when the data exceeds the main memory. Conversely, while standard parallel algorithms provide massive speedups, they cause catastrophic I/O bottlenecks when applied to external memory environments.

This thesis successfully addressed this gap by designing a unified approach for the Parallel External Memory (PEM) model. We proposed the **Tiled Wavefront Algorithm**, which elegantly combines the I/O efficiency of sequential blocked DP with the computational speedup of parallel wavefront scheduling.

By elevating the wavefront schedule from a fine-grained, cell-by-cell level to a coarse-grained, tile-by-tile level, the algorithm ensures that each processor is saturated with $O(M)$ local computation for every $O(\sqrt{M}/B)$ blocks fetched from the disk. Furthermore, by carefully structuring the logical grid and overlapping tile boundaries, the algorithm strictly adheres to the Concurrent Read Exclusive Write (CREW) constraints of the PEM model,

ensuring zero write conflicts in shared memory.

Finally, using the WT Scheduling Principle (Brent’s Theorem), we rigorously derived the parallel I/O complexity of the proposed algorithm:

$$T_P = O\left(\frac{N^2}{PB\sqrt{M}} + \frac{N}{B}\right)$$

This bound theoretically proves that the Tiled Wavefront Algorithm achieves an optimal balance, scaling perfectly with the number of processors (P) and the block size (B), while successfully using the internal memory limit (M) to completely eliminate disk thrashing.

5.2 Future Work

While this thesis establishes a solid theoretical foundation and mathematically proves the optimality of the Tiled Wavefront Algorithm, several promising avenues remain for future research and practical extension:

- **Empirical Implementation and Benchmarking:** The most immediate next step is to transition this algorithm from a theoretical model to a practical implementation. Future work should involve implementing the algorithm using parallel programming frameworks (such as OpenMP or MPI) and testing it on a modern multi-core cluster connected to a high-speed SSD array. Benchmarking against standard PRAM implementations will provide concrete empirical evidence of the I/O savings.
- **Space Optimization (Hirschberg’s Algorithm):** The current dynamic programming approach requires $O(N^2)$ space to reconstruct the actual longest common subsequence (rather than just finding its length). Future research can explore adapting Hirschberg’s divide-and-conquer strategy[Hir75] to the PEM model. This would theoretically reduce the external memory space requirement to $O(N)$ while maintaining the parallel I/O efficiency.

- **Cache-Oblivious Adaptation:** The proposed algorithm is "cache-aware," meaning its performance depends on tuning the tile size S to match the specific hardware parameters M and B . Future work could investigate designing a *Cache-Oblivious* version[CR06] of this parallel algorithm, which achieves optimal I/O block transfers automatically without needing to know the exact sizes of the memory hierarchy levels in advance.
- **Extension to Similar Sequence Problems:** The mathematical principles and wavefront scheduling techniques developed in this thesis are highly versatile. They can be extended and generalized to solve other massive-scale DP problems, such as the Edit Distance (Levenshtein distance) problem or the Smith-Waterman algorithm for local sequence alignment, providing a broader impact on computational biology.

References

- [AHU74] A. V. Aho, J. E. Hopcroft, and J. D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.
- [AV88] A. Aggarwal and J. S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [CLRS09] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.
- [CR06] Rezaul A. Chowdhury and Vijaya Ramachandran. Cache-oblivious dynamic programming. In *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 551–560, 2006.
- [Hir75] D. S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6):341–343, 1975.
- [Já92] J. JáJá. *An Introduction to Parallel Algorithms*. Addison-Wesley, 1992.
- [SPS23] G. Sajith, M. K. Panda, and R. Senko. Sorting on the parallel random access machine, parallel comparison model and parallel external memory model, 2023.